Ethics and Legality Integration into the Agile Software Development Pipeline

Social and Ethical Issues

University of Pisa
June 5, 2026

Contents

1 Introduction

Modern software systems increasingly operate as socio-technical infrastructures: they do not merely execute functions, but mediate access to services, process personal data, automate decisions, and shape organisational behaviour. For this reason, a software pipeline cannot be evaluated only in terms of functional correctness, delivery speed, or security. It must also make ethical values and legal obligations explicit, traceable, testable, and monitorable.

This report proposes a progressive engineering model for integrating ethics and legality into a contemporary software development DevOps pipeline. The model starts from an Agile sprint loop in which the most of the backward arrows are implicit because of the iterative nature of Agile development and to simplify the diagram.

The Agile loop is expanded through four layers:

1. **Agile based on Value Sensitive Design (VSD)** [**?**, Ch. 13]: values and stakeholders enter the backlog.
2. **Conceptual Engineering (CE) with value hierarchy** [**?**, Secs. 2–4]: ambiguous values are clarified before becoming requirements transforming them into concepts.
3. **GORE + SLEEC** [**?**, Secs. 3–4]: values become normative goals and then formal, testable rules.
4. **MLOps + Compliance-as-Code + Runtime Assurance** [**?**, Sec. Implementing Ethics in the Development Pipeline], [**?**, Secs. 3–4], [**?**, Secs. 4–5], [**?**, Secs. 3–5], [**?**, Secs. 5.2.3–5.2.5 and 9.2.3]: ethical and legal requirements are enforced through CI/CD gates, policy engines, monitoring, and human oversight.

The central thesis is that responsible software engineering requires a shift from *late ethical review* to *continuous ethical and legal assurance*. This also reframes responsibility into two dimensions: **Active responsibility** (forward-looking): the engineering organisation designs, tests, monitors, and documents systems to prevent foreseeable harm; **Passive responsibility** (backward-looking): when failures occur, the organisation must reconstruct who decided what, which evidence was available, which controls were applied, and why the deployed system behaved as it did.

2 Conceptual Foundations

2.1 Cybernetics: feedback as the organising principle

The proposed pipeline is cybernetic in structure [**?**, Ch. 4]. It is organised around feedback loops that compare observed behaviour with desired goals and then correct the system or the process. Agile retrospectives, CI test failures, policy violations, model drift alerts, incident postmortems, and human override mechanisms all instantiate negative feedback.

This matters because ethical and legal compliance cannot be treated as a static checklist. Values evolve, stakeholders reinterpret impacts, operational environments change, and AI models degrade or behave unexpectedly. Therefore, the pipeline must repeatedly evaluate also if the system still satisfies its goals.

2.2 The limits of the intelligence imitation of modern machines

The Turing-test tradition frames intelligence behaviourally [**?**, Sec. 1]: a machine may appear intelligent if its outputs are indistinguishable from those of a human. However, software engineering for responsible AI cannot rely only on output imitation. A system may produce convincing behaviour while still violating privacy, amplifying bias, or hiding causal responsibility. The distinction between weak and strong AI is therefore operationally relevant: most deployed AI systems should be engineered as powerful decision-support artefacts, not as autonomous moral subjects.

Also the Searle's Chinese Room argument is useful as a warning for formal compliance [**?**, pp. 417–420]. A rule engine may manipulate symbols correctly while lacking semantic understanding of the underlying value. This is not an argument against formalisation; rather, it shows why formal rules must remain connected to stakeholder interpretation, domain expertise, legal review, and human oversight. In the proposed pipeline, SLEEC rules and policy-as-code are therefore treated as *engineering artefacts requiring governance*, not as replacements for judgement.

2.3 Responsibility and the Problem of Many Hands

Active responsibility becomes operational through blocking controls: the pipeline prevents non-compliant artefacts from reaching production. **Passive responsibility** becomes operational through evidence: every policy decision, exception, deployment, and runtime alert is logged for later audit and accountability.

This directly addresses the many-hands problem. Instead of relying on memory or informal coordination, the pipeline records decisions and makes responsibility attributable at the artefact level.

Modern pipelines distribute action across product owners, developers, data engineers, security engineers, ML engineers, legal teams, platform teams, policy engines, and runtime systems. This generates the classic problem of "many hands": outcomes are collectively produced, but accountability can become diffuse. In AI systems, this may create a **responsibility gap**, especially when decisions emerge from blackbox models from which is hard to identify a responsible party [**?**, pp. 175–182].

The pipeline proposed in this report addresses the responsibility gap by introducing an explicit **RACI Matrix** after the definition of the Sprint Backlog and before the CI/CD pipeline. Once the value-derived norms, GORE/SLEEC rules, technical tasks, and compliance controls have been selected for the sprint, the RACI Matrix assigns clear responsibility roles to the relevant actors:

- **Responsible**: the actor who performs the task or implements the control;
- **Accountable**: the actor who owns the final decision or approval;
- **Consulted**: the actors whose expertise is required, such as legal, ethics, domain, or security experts;
- **Informed**: the actors who must be kept aware of decisions, failures, exceptions, or residual risks.

In this way, responsibility is not left implicit in the technical pipeline. The RACI Matrix connects requirements, implementation tasks, SLEEC-derived rules, CaC policies, quality gates, audit evidence, and runtime assurance mechanisms to identifiable roles. This reduces the risk that accountability becomes diffuse across the many actors involved in the development and operation of an AI-enabled software system.

3 Value Sensitive Design in Agile

3.1 The Agile loop

The initial diagram is a standard Agile loop, of which an example can be found in Figure 1. Its strength is rapid iteration, while its weakness is that values may remain implicit unless they are deliberately introduced into sprint artefacts. For this reason, the backlog is not treated only as a queue of features: it is the first place where affected stakeholders, data sources, legal constraints, and foreseeable harms must become visible engineering inputs.

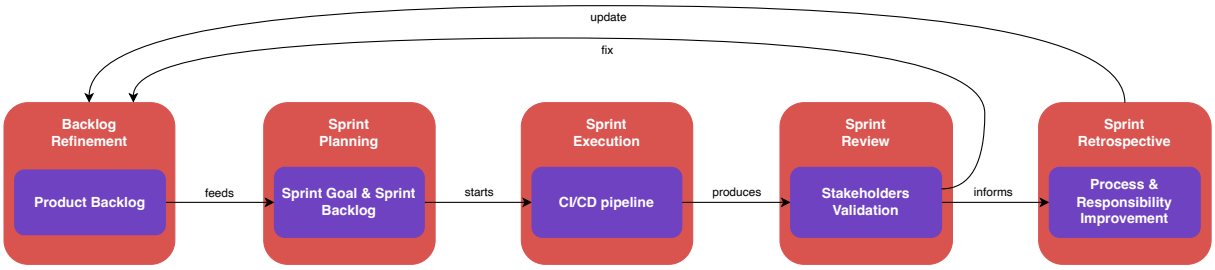

Figure 1: Baseline Agile/Scrum sprint loop. The loop connects backlog refinement, sprint planning, sprint execution, sprint review, and sprint retrospective through delivery and feedback flows.

3.2 Stakeholder tokens and value source analysis

Within the proposed pipeline, stakeholder tokens and value source analysis are both performed inside the *Value Elicitation* activity of the backlog refinement phase. This activity is not intended to produce final technical requirements directly. Rather, it enriches the backlog with an explicit understanding of the actors affected by the system and of the values that should constrain the resulting user stories.

Stakeholder tokens make affected actors visible during discussion, preventing the team from considering only the primary user or the customer. In this report, they are interpreted as a practical way to keep track of direct users, data subjects, maintainers, regulators, vulnerable groups, non-users, and indirectly affected communities while backlog items are being refined.

Value source analysis complements this by clarifying where the relevant values originate. These sources may include stakeholder expectations, legal requirements, organisational principles, professional codes, domain standards, social expectations, and risk assessments. In practice, stakeholder tokens help identify whose interests must be represented, while value source analysis explains why values such as privacy, fairness, transparency, or autonomy should constrain specific user stories.

The output of *Value Elicitation* is therefore a set of values associated with the backlog which are transformed into norms (as we'll see in the next chapters) and then into user stories. Then they can be clarified through Conceptual Engineering and refined into GORE/SLEEC requirements.

3.3 Conceptual Engineering

A value such as privacy, autonomy, transparency, or fairness is too abstract to be directly implemented; so, different conceptualisations of the same value can imply different technical requirements. The Design for Values and Conceptual Engineering literature argues that value-based requirements engineering needs systematic conceptual work because different conceptions of abstract values lead to different design requirements [**?**, Secs. 2–4]. This is also where the pipeline protects itself against purely syntactic compliance: a term may be accepted by a rule or a ticket template while still being semantically unclear for stakeholders, regulators, or operators.

During sprint planning, the team should run a short conceptual clarification step:

1. **Discovery**: how is the value currently understood by stakeholders and in the domain?
2. **Justification**: does the current concept serve the intended moral, legal, and engineering function?
3. **Construction**: which refined conception should be adopted for this system?

In the integrated diagrams, this is represented as a CE loop inside sprint planning. The loop is deliberately placed before the sprint backlog is fixed: discovery gathers how a value is currently used, justification checks whether that use serves the moral, legal, and engineering purpose, and construction records the clarified concept that the team will implement. This prevents ambiguous values from silently becoming ambiguous requirements.

3.4 First value hierarchy

The first stable artefact is a value hierarchy. It connects each abstract value to a more concrete norm, then to a design requirement, and finally to a verification artefact that can be checked during development.

For GDPR-relevant systems, this hierarchy should be aligned with Article 5 principles such as lawfulness, fairness, transparency, purpose limitation, data minimisation, accuracy, storage limitation, integrity/confidentiality, and accountability, and with Article 25 data protection by design and by default [**?**, Art. 5], [**?**, Art. 25], [**?**, Secs. 2–3]. For AI systems, it should also be aligned with the AI Act's risk-based framework and requirements such as human oversight for high-risk AI systems [**?**, Sec. Risk-based approach], [**?**, Art. 14].

3.5 Integration with the Agile model

Value Sensitive Design and Conceptual Engineering are useful here because are not an external ethical audit added after implementation. It is a design approach that incorporates human values throughout design work. The Agile-VSD literature argues that VSD can be integrated into existing Agile workflows rather than replacing them; values can be represented and refined

Value	Norm	Requirement	Verification
Privacy	Data minimisation	Collect only fields required for the declared purpose.	Schema test; privacy review
Autonomy	Meaningful user control	Provide users with the ability to change their choices or appeal decisions.	UX test; audit log
Fairness	Non-discrimination	Evaluate model performance across relevant groups.	Fairness test report
Transparency	Explainability	Provide understandable reasons for consequential decisions.	Explanation acceptance test
Security	Integrity and confidentiality	Encrypt personal data at rest and in transit.	IaC/policy test

Table 1: Some examples of value hierarchy.

inside sprint planning, model evaluation and backlog management [**?**, Secs. 13.4–13.5]. Figure 2 shows this integration by connecting stakeholder analysis, value elicitation, value-sensitive implementation, testing, review, and retrospective updates to the normal sprint loop. The added AI risk and data-source inputs make the pipeline sensitive to a central problem of data-driven systems: behaviour may look correct while the underlying data selection, measurement categories, or optimisation target still encode bias, opacity, or privacy risk.

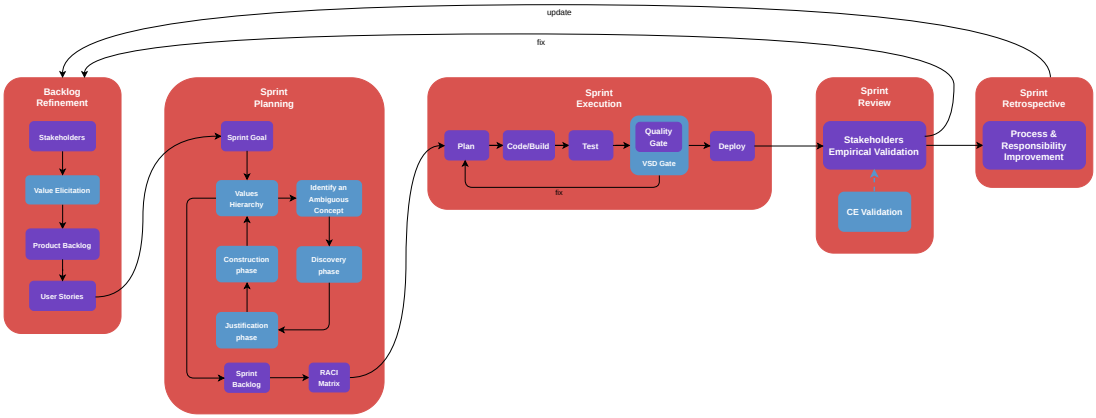

Figure 2: Agile/Scrum sprint loop enriched by a Value Sensitive Design layer. VSD activities feed the backlog, planning, execution, and review points instead of being treated as a separate downstream phase.

4 Requirements Engineering through GORE and SLEEC

4.1 From values to Norms

The next expansion of the model moves from value analysis to requirements engineering. Goal-Oriented Requirements Engineering (GORE) is used to represent stakeholder intentions as goals. In the ethics-aware adaptation literature, ethical values are represented as **normative goals** aligned with functional and adaptation goals, enabling analysis of trade-offs and conflicts [**?**, Sec. 3]. In this stage, values are transformed into normative goals, connected to functional or adaptation goals, decomposed into tasks, and finally expressed as rules that can be reviewed and tested. GORE gives the pipeline a place to ask why a requirement exists, not only what the system must do; this matters when competing goals such as privacy, safety, usability, and accountability cannot all be maximised at the same time.

4.2 GORE artefacts

A practical GORE model for this report should distinguish:

- **Normative goals**: obligations grounded in values, law, or institutional principles.
- **Functional goals**: services the system must provide.
- **Adaptation goals**: how the system changes behaviour under context changes.
- **Obstacles**: conditions that prevent goal satisfaction.
- **Trade-offs**: when there are conflicts between goals (e.g. privacy versus safety).

4.3 From normative goals to SLEEC rules

SLEEC rules translate normative goals into a form that is close to natural language but structured enough for automated analysis. In the cited SLEEC work, rules have the general form "`when <condition> then <response> [within <time>] [unless <exception>]`" [**?**, Sec. 4]. In the pipeline, SLEEC is the bridge between ethical language and executable control: it keeps the rule understandable for human review while making triggers, responses, deadlines, and exceptions precise enough for tests and monitors.

Listing 1: Example SLEEC-style rule for GDPR deletion.

```
rule delete_personal_data:
  when UserRequestsErasure and requestIsValid
  then DeletePersonalData within 30 days
  unless legalRetentionObligation
  then RestrictProcessing and NotifyUser within 7 days
```

Listing 2: Example SLEEC-style rule for human oversight.

```
rule human_review_high_risk_decision:
  when HighRiskAIDecisionGenerated and confidence < threshold
  then EscalateToHumanReviewer within 5 minutes
```

4.4 Testability and conflict detection

SLEEC-style rules improve engineering quality because they support well-formedness checking, conflict detection between rules, mapping to acceptance tests, mapping to runtime monitors, and review by non-technical stakeholders. In fact it can be viewed as a bridge between the ethical language of values and the technical language of code, allowing a better communication between developers and stakeholders.

This stage is where values become technically actionable. However, the Chinese Room argument still applies: formal rules must remain explainable and reviewable, because syntactic rule execution does not guarantee semantic adequacy.

4.5 Integration with the Agile-VSD model

Figure 3 shows how the Agile-VSD loop is extended with a requirement engineering branch. Conceptual Engineering clarifies values during sprint planning, GORE turns them into normative goals, and SLEEC rules make them precise enough to feed the sprint backlog. The diagram therefore makes responsibility assignable: each value is transformed into a concept (CE), each concept into one or more normative goals (GORE), and for each one a rule is written (SLEEC).

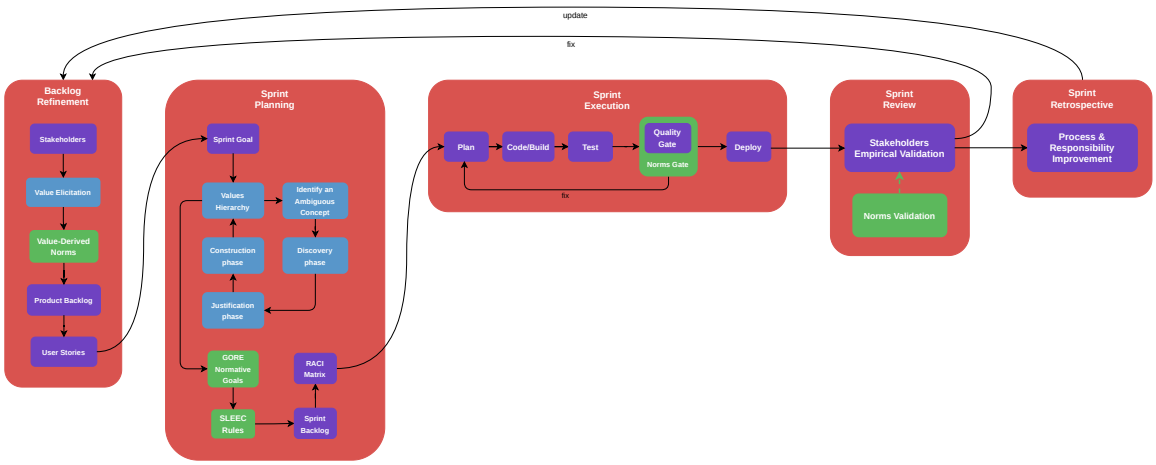

Figure 3: Detailed Agile-VSD planning view with Conceptual Engineering, GORE normative goals, SLEEC rules, and sprint backlog formation.

5 DevOps and Compliance-as-Code

5.1 From ethical requirements to pipeline controls

The next expansion shifts the focus from requirements specification to the DevOps CI/CD pipeline. Compliance-as-Code (CaC) expresses legal and organisational compliance requirements as executable, version-controlled, testable policy artefacts. The CaC literature describes this as

a shift from document-centric and retrospective compliance toward programmatic controls embedded directly in the software delivery lifecycle [**?**, Secs. 2–4], [**?**, Secs. 3–5].

5.2 Shift-left compliance

Shift-left compliance means that checks are executed as early as possible, ideally before code reaches production. CaC should be integrated at multiple points, as defined in Table 2.

The Deployflow source describes an audit-ready DevOps pipeline as a pipeline that does not rely on manual evidence collection after the fact. Instead, it automatically records relevant actions, preserves tamper-resistant evidence, controls access through Role-Based Access Control (RBAC), enforces security measures such as encryption, monitors the system continuously, and produces audit reports from the information already collected during development and deployment. In this sense, the pipeline makes each change repeatable, traceable, and auditable [**?**, Sec. Audit-Ready Pipelines in Action]. The EthSecDevOps article published by BML Global reinforces the same direction from an ethical perspective: ethics should be treated as a first-class concern, with early value assessment, stakeholder analysis, ethical code reviews, fairness testing, ethical deployment checklists, and continuous monitoring [**?**, Secs. Implementing Ethics in the Development Pipeline and Deployment and Operations].

Pipeline stage	**Typical controls**	**Evidence produced**
Code / pre-commit	secret detection, linting, privacy annotations, prohibited API checks	commit metadata, static-analysis report
Build	dependency scanning, licence checks, reproducible build	SBOM, dependency report, build signature
Test	privacy tests, fairness tests, security tests, SLEEC-derived acceptance tests	test report, model-card update, risk record
Release gate	OPA/Sentinel policy evaluation, approval workflow, exception handling	policy decision log, approver identity
Deploy	infrastructure-as-code validation, RBAC, encryption, network controls	deployment manifest, IaC compliance report
Operate	runtime monitoring, audit logs, drift detection, incident alerts	immutable logs, alert history, incident timeline

Table 2: Compliance controls across the CI/CD pipeline.

5.3 Policy-as-Code tools

Policy-as-Code tools such as Open Policy Agent (OPA), HashiCorp Sentinel, Chef InSpec, and Cloud Custodian can express compliance constraints in executable form. In the model, these tools

populate the CaC library rather than acting as isolated scripts. A retention rule, an encryption rule, or an access-control rule becomes part of a governed repository that can be tested, reviewed, versioned, and connected to audit evidence.

Listing 3: Illustrative OPA/Rego-style rule.

```
deny[msg] {
  input.resource.type == "database"
  not input.resource.encryption_at_rest
  msg := "Database must enable encryption at rest."
}
```

Listing 4: Illustrative retention rule.

```
deny[msg] {
  input.dataset.contains_personal_data
  input.dataset.retention_days > input.policy.max_retention_days
  msg := "Personal-data retention exceeds the approved policy."
}
```

5.4 Integration with the Agile Normative-based model

The previous requirements branch is connected to CI/CD gates. Normative goals and SLEEC rules become policy-as-code checks, those checks block or approve pipeline stages, and their decisions are stored as audit evidence. The CaC library in the diagram is the versioned memory of these rules: it prevents compliance from depending on informal recollection and makes policy changes reviewable like source code. The compliance gate is the point where the pipeline acts, because a failed legal, security, or ethical control can stop a release before harm reaches production. The audit evidence store gives the same process a backward-looking function, preserving policy decisions, exceptions, and deployment records.

6 MLOps and Runtime Assurance

6.1 Why AI requires a further expansion

AI systems add new sources of risk: training data may be biased, model behaviour may be statistically uncertain, the operational environment may diverge from the training environment, and performance may degrade after deployment. The responsible-AI software engineering roadmap highlights that responsible AI must be operationalised across governance, process, and system perspectives, not only at the algorithm level [**?**, Secs. 3–5]. It also emphasises continuous and iterative monitoring, co-versioning of data/model/code/configuration, audit trails, and dynamic ethical risk assessment during operation [**?**, Sec. 4.1]. The AI risk and data-source block in the diagram expresses this shift: responsible AI begins before model training, when the team decides which data sources are legitimate, which affected groups may be represented or excluded, and which risks must enter the backlog.

For AI systems, the DevOps pipeline must also manage data, training, validation, deployment, runtime monitoring, and possible retraining or rollback.

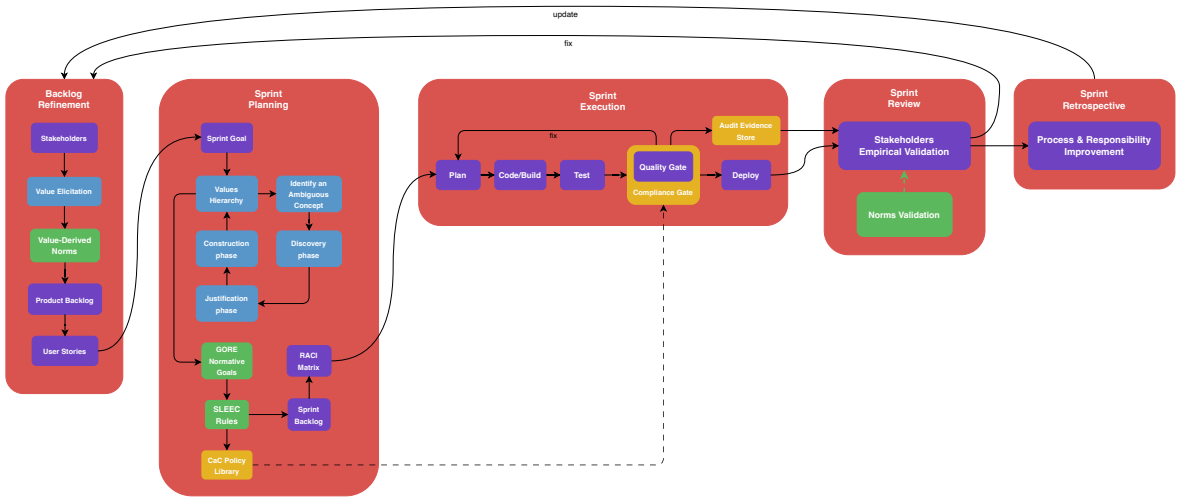

Figure 4: CI/CD expansion with Compliance-as-Code gates and audit evidence. Compliance policy evaluation is placed inside the build-test-deploy flow, and audit evidence is stored as part of the delivery process.

6.2 The W-shaped AI lifecycle

For AI systems, especially high-risk systems, the W-shaped lifecycle is more appropriate than a simple V-model. The TAID white paper describes the AI W-shape as an expansion of the traditional V-model that accounts for the specificities of AI constituents, including the tight connection between the AI model and its pre/post-processing [**?**, Sec. 5.2.3]. It distinguishes the *host platform*, where the AI model and software system are developed, trained, tested, and validated before deployment, from the *target platform*, where the deployed AI-enabled system operates under real runtime conditions and requires monitoring, human oversight, and assurance mechanisms [**?**, Secs. 5.2.4–5.2.5].

6.3 Design-time assurance

As shown in Appendix Figure **??**, Design-time assurance takes place before deployment. Its purpose is to check whether the AI component is sufficiently reliable, compliant, and fit for the operational context in which it will be used. It includes checks on data quality and lineage, bias and representativeness, model validation, robustness and generalisation, explainability, and conformity with legal or organisational policies.

The key additional control is the Model Evaluation Gate. This gate ensures that a trained model is not accepted simply because it can be executed, but only if there is enough evidence that it satisfies its requirements and that the remaining risks are understood.

In this sense, design-time assurance mainly supports active responsibility: foreseeable harms must be identified, assessed, and mitigated before the model is deployed.

6.4 Runtime assurance

Runtime assurance is performed after deployment and focuses on whether the deployed system remains trustworthy under actual operating conditions. Appendix Figure **??** separates this runtime layer from the design-time gates and shows the monitoring and control mechanisms that remain active after deployment. Data drift monitoring checks whether input distributions have changed, concept drift monitoring checks whether the relation between inputs and outputs has shifted, and performance monitoring follows accuracy, latency, error rates, and fairness metrics. Safety-envelope monitoring detects states outside the approved operational domain, while fallback mechanisms, rollback, conservative decision logic, and human escalation turn monitoring into corrective feedback rather than passive observation.

Runtime assurance supports both active and passive responsibility. It actively prevents uncontrolled degradation and passively produces evidence for incident reconstruction.

6.5 Meaningful Human Control

The Meaningful Human Control (MHC) is the mechanism that prevents technological autonomy from eliminating human accountability. The TAID white paper identifies MHC as a key open issue for trustworthy AI and links it to compliance, dignity, and responsibility [**?**, Secs. 6.3 and 7.1.4]. In the pipeline, MHC should not be a vague statement. It must be engineered through explicit thresholds for human escalation, user interfaces that provide context and explanation, authority to override or stop the system, logging of human decisions and machine recommendations, and clear training and role definitions for operators.

6.6 Final Agile Normative & Risk-prevention based model

In Figure **??**, the AI risk and data-source inputs enter backlog refinement before sprint planning, so the system starts from the people, data, and harms that may be affected. Sprint planning then contains the CE loop, where ambiguous values are clarified before they become engineering commitments. GORE and SLEEC translate those commitments into goals and rules, the CaC library stores the executable policy version of those rules, and the compliance gate checks them during delivery. The audit evidence store preserves the decisions produced by these gates, while the model evaluation gate and runtime assurance blocks keep the AI component under control before and after deployment.

7 Conclusions

The final pipeline transforms ethics and legality from abstract external concerns into engineering artefacts. Agile provides the iterative structure; VSD introduces stakeholders and values; Conceptual Engineering clarifies the meaning of those values; GORE and SLEEC operationalise them as goals and rules; Compliance-as-Code enforces them in the CI/CD pipeline; MLOps and Runtime Assurance keep AI systems aligned after deployment.

The result is a pipeline oriented toward **socio-technical trustworthiness**. It does not guarantee perfect ethical behaviour, but it creates a disciplined framework for making values explicit,

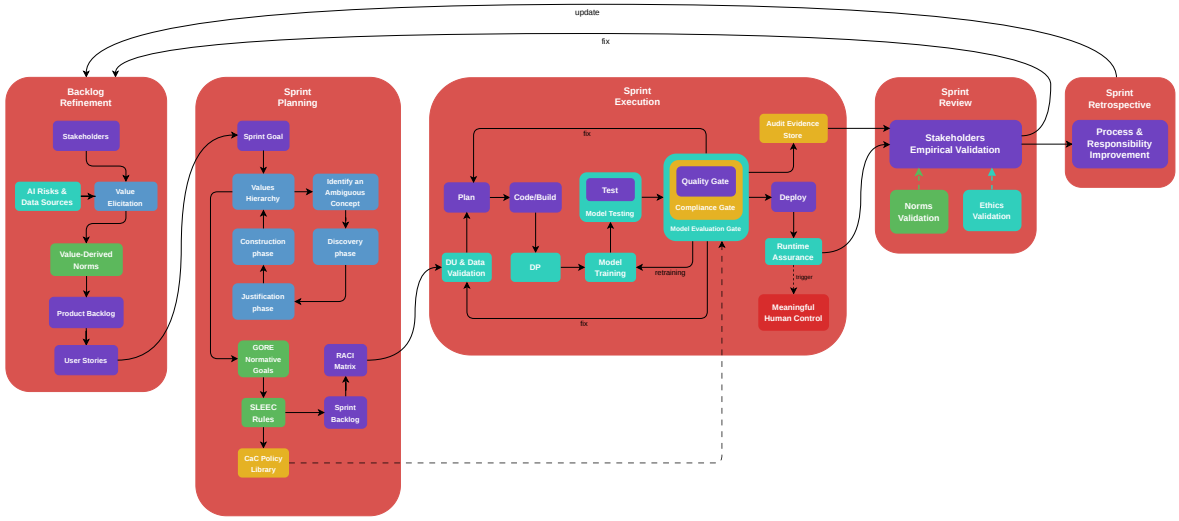

Figure 5: Final integrated pipeline. The complete model connects VSD, Conceptual Engineering, GORE/SLEEC, Compliance-as-Code, MLOps, evidence storage, model registry, explainability, and human oversight.

legal obligations traceable, compliance testable, runtime behaviour observable, and responsibility attributable.

A MLOps Assurance Diagram

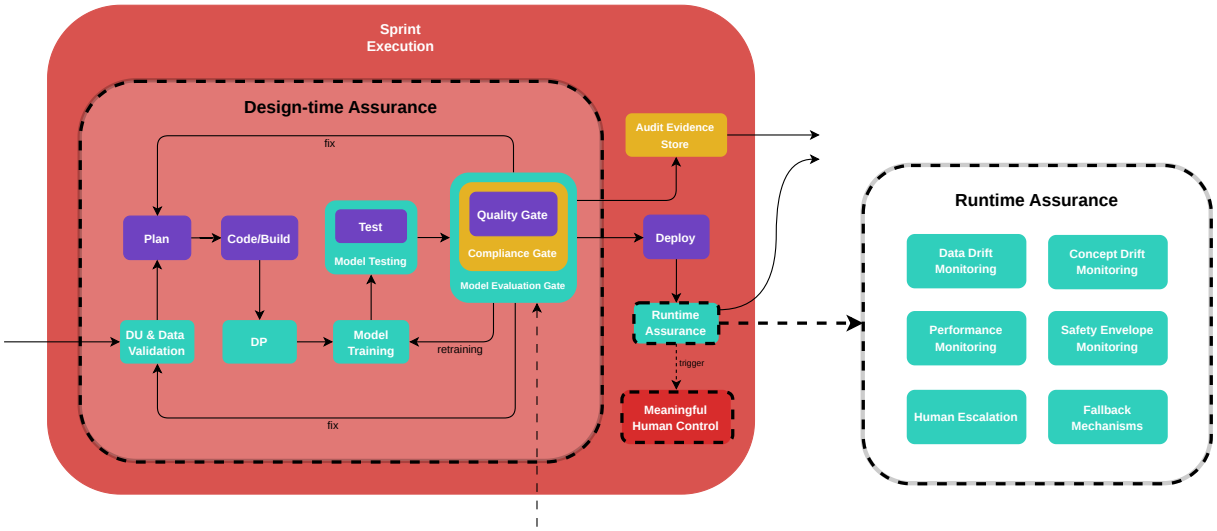

Figure 6: MLOps assurance view. Design-time assurance covers data validation, model training, model testing, model evaluation gates, compliance gates, and evidence collection before deployment; runtime assurance monitors drift, performance, safety-envelope violations, fallback mechanisms, and human escalation after deployment.

B Traceability matrix

Table 3: End-to-end traceability from value to operational evidence.

Level	Artefact	Example	Evidence
Value	Value statement	Privacy must be preserved	Stakeholder workshop notes
Concept	Clarified concept	Privacy as minimisation, control, confidentiality	Conceptual decision record
Norm	Normative statement	Only necessary data may be processed	Legal mapping to GDPR Art. 5/25
Goal	Normative goal	Minimise personal-data collection	GORE model
Rule	SLEEC rule	When unnecessary field is requested, block collection	Rule repository
Policy	Policy-as-code	Deny schema with unapproved PII	OPA/Sentinel decision log
Test	CI/CD test	Privacy regression test	CI report

Runtime	Monitor	Retention/drift/ access alert	Immutable audit log
Governance	Review	Human approval or exception	Approval record and rationale

C Mapping Course Topics to the Pipeline

Table 4: Connections with lecture topics.

Lecture topic	**Pipeline connection**
Cybernetics and behaviour	Agile, CI/CD, runtime monitoring, and human oversight are feedback loops that correct deviations from intended goals.
Machine intelligence	The pipeline treats AI as engineered decision-support systems whose outputs require assurance, not as moral agents.
Philosophy of mind	The Chinese Room motivates human review and semantic validation of formal rules.
AI ethics and machine ethics	The pipeline distinguishes ethics for creators from ethical behaviour constraints for machines.
Big data and algorithms	Bias, opacity, explainability, data lineage, and drift are treated as testable and monitorable properties.
Responsibility	Active responsibility maps to prevention; passive responsibility maps to auditability, incident reconstruction, and accountability.
Legal framework and CSR	GDPR, AI Act, CSR, and organisational governance become pipeline artefacts rather than external documents.

D Implementation Blueprint

A practical implementation can proceed incrementally:

1. **Sprint 0**: define stakeholders, value sources, legal scope, and initial value hierarchy.
2. **Sprint 1**: convert the most important values into GORE normative goals and SLEEC-style rules.
3. **Sprint 2**: implement CI checks for the highest-risk GDPR controls: encryption, retention, access control, logging, and data minimisation.
4. **Sprint 3**: add policy-as-code gates with OPA or Sentinel.
5. **Sprint 4**: add audit evidence storage and dashboarding.
6. **Sprint 5**: for AI systems, add model registry, dataset lineage, model cards, fairness tests, and drift monitors.

7. **Sprint 6**: formalise human oversight, exception handling, and incident response procedures.

References

[1] N. Wiener, *Cybernetics: Or Control and Communication in the Animal and the Machine*, MIT Press, 1948.

[2] A. M. Turing, "Computing Machinery and Intelligence," *Mind*, vol. 59, no. 236, pp. 433–460, 1950. DOI: https://doi.org/10.1093/mind/LIX.236.433.

[3] J. R. Searle, "Minds, Brains, and Programs," *Behavioral and Brain Sciences*, vol. 3, no. 3, pp. 417–424, 1980. DOI: https://doi.org/10.1017/S0140525X00005756.

[4] A. Matthias, "The responsibility gap: Ascribing responsibility for the actions of learning automata," *Ethics and Information Technology*, vol. 6, pp. 175–183, 2004. DOI: https://doi.org/10.1007/s10676-004-3422-1.

[5] S. Umbrello and O. Gambelin, *Agile as a Vehicle for Values: A Value Sensitive Design Toolkit*, preprint version, later published in Springer, 2023.

[6] H. Veluwenkamp and J. van den Hoven, "Design for values and conceptual engineering," *Ethics and Information Technology*, vol. 25, article 2, 2023. DOI: https://doi.org/10.1007/s10676-022-09675-6.

[7] E. Silva Junior, L. Marsso, R. Caldas, M. Chechik, and G. N. Rodrigues, *Operationalizing Human Values in the Requirements Engineering Process of Ethics-Aware Autonomous Systems*, arXiv:2602.09921, 2026.

[8] Q. Lu, L. Zhu, X. Xu, J. Whittle, and Z. Xing, *Towards a Roadmap on Software Engineering for Responsible AI*, arXiv:2203.08594, 2022.

[9] P. Singh, "Integrating Compliance-as-Code into CI/CD Pipelines for Regulated Industries," *International Journal of Innovative Research and Creative Technology*, vol. 11, no. 2, 2025.

[10] L. Potharalanka, "Compliance-As-Code: A Framework for Regulatory Automation in Enterprise CI/CD Pipelines," *Sarcouncil Journal of Engineering and Computer Sciences*, vol. 4, issue 7, pp. 257–267, 2025.

[11] European Defence Agency, *Trustworthiness for AI in Defence: Developing Responsible, Ethical, and Trustworthy AI Systems for European Defence*, TAID White Paper, version 1.0, 09/05/2025.

[12] European Defence Agency, *Trustworthiness for AI in Defence*, sections on Meaningful Human Control and Value-Based Engineering, 2025.

[13] D. Webster and B. Johnson, "EthSecDevOps: Integrating Ethics as a First-Class Citizen in Product Development," BML Global. Available at: https://bml.global/ethsecdevops-integrating-ethics-as-a-first-class-citizen-in-product-development/.

[14] N. Ilic, "Making GDPR Practical: Real-World DevOps Pipelines That Pass Audits," Deployflow, 2025. Available at: https://deployflow.co/blog/making-gdpr-practical-devops-pipelines-pass-audits/.

[15] European Union, General Data Protection Regulation, Article 5, "Principles relating to processing of personal data." Available at: https://gdpr-info.eu/art-5-gdpr/.

[16] European Union, General Data Protection Regulation, Article 25, "Data protection by design and by default." Available at: https://gdpr-info.eu/art-25-gdpr/.

[17] European Data Protection Board, *Guidelines 4/2019 on Article 25 Data Protection by Design and by Default*, version 2.0, 2020.

[18] European Commission, "AI Act," Shaping Europe's Digital Future. Available at: https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai.

[19] European Union, Artificial Intelligence Act, Article 14, "Human oversight." Available at: https://artificialintelligenceact.eu/article/14/.